



VyaparMind Technical Documentation ■■

Comprehensive Documentation Suite



Developer Guide & System Architecture

1. System Architecture

VyaparMind uses a **Monolithic Modular Architecture** built on Python and Streamlit.

Tech Stack

- **Frontend/App Layer:** [`Streamlit`](https://streamlit.io/) (v1.38+)
- **Database:** `SQLite3` (Native Python support)
- **Data Processing:** `Pandas` (DataFrames for tabular logic)
- **Visualization:** `Plotly` (Interactive charts)
- **Reporting:** `ReportLab` (PDF Generation)
- **Integrations:**
 - `Twilio` (WhatsApp API - Simulated)
 - `SMTP` (Email - Simulated)

Directory Structure

`

/rpworkspace/VyaparMind/

- app.py # (Entry Point) Login, Route Handling, Global Styles
- database.py # (Core) DB Connection, CRUD Functions, Schema Init
- config.py # Global Constants (Account IDs, Paths)
- ui_components.py # Reusable UI widgets (Headers, Sidebar, Auth Checks)
- whatsapp_utils.py # Alerting Logic
- pdf_utils.py # Invoice & Report Generation
- email_utils.py # Email Templates & Sending Logic
- pages/ # (Modules) auto-routed by Streamlit
 - ■■■ 1_Inventory.py
 - ■■■ 2_POS.py

■ ■ ■ ■ 3_Settings.py

■ ■ ■ ■ ... (25+ Modules)

■ ■ ■ ■ 99_SuperAdmin.py

■ ■ ■ assets/ # Static files (Logos, CSS)

,

2. Database Schema (Key Tables)

The system uses a relational SQLite database.

■ Tenant / Core

- **`accounts`**: Multi-tenant isolation.
- **`id`** (PK), **`company\_name`**, **`subscription\_plan`**, **`status`**
- **`users`**: Staff accounts.
- **`id`**, **`account\_id`** (FK), **`username`**, **`password\_hash`**, **`role`**
- **`settings`**: Key-Value store for account config.

❄ ■ Cold Storage

- **`cold\_zones`**: Temperature Zones.
- **`target\_temp\_min`**, **`target\_temp\_max`**, **`capacity\_pallets`**
- **`storage\_clients`**: Customer CRM.
- **`rate\_card\_json`** (Stores custom billing logic)
- **`inward\_receipts`** (GRN): Heads of Inward process.
- **`grn\_number`**, **`quality\_status`** ('PENDING'/'APPROVED')
- **`cold\_inventory`**: The central stock table.
- **`client\_id`** (FK), **`lot\_number`**, **`expiry\_date`**, **`zone\_id`** (FK)

■ Retail

- **`products`**: Item master.

- ``transactions``: Sales headers.
- ``transaction_items``: Line items linked to transactions.

3. Key Utility Modules

``database.py`` (The Backend)

This file is the backbone of the application. It handles:

- **Connection**: ``get_connection()`` ensures thread-safe SQLite access.
- **WAL Mode**: Write-Ahead Logging is enabled for concurrency performance.
- **Functions**: e.g., ``create_tenant()``, ``log_temperature()``, ``approve_grn_quality()``.

``whatsapp_utils.py`` (The Alert System)

- Defines specific message templates for:
- Temperature Breaches (Manager Alert)
- Delivery Confirmations (Client Alert)
- Expiry Warnings
- Currently set to **DEMO MODE** (prints to console). To go live, add Twilio credentials in ``env``.

4. Setup & Deployment

Local Development

1. Install Requirements:

```
`bash
```

```
pip install streamlit pandas plotly reportlab
```

```
`
```

2. Run Application:

```
`bash
```

streamlit run app.py

,

Server Deployment

- Recommended: **Docker** or **VPS (Ubuntu 22.04)**.
- Process Manager: Use ``Supervisor`` or ``Systemd`` to keep Streamlit running.
- Web Server: Use **Nginx** as a reverse proxy for SSL (HTTPS).

5. Security & Access Control

- **Role Based Access Control (RBAC):**
 - **Super Admin:** Full System Access (Tenant Creation).
 - **Admin:** Tenant-level full access.
 - **Manager:** Can edit data but not users.
 - **Staff:** Read/Write operational data.
- **Data Isolation:** Every DB query MUST filter by ``account_id`` (``get_current_account_id()``).

End of Technical Documentation